PHASE 2 – Deep Learning (Days 11–14)

Neural Networks

Neural networks are machine learning models that mimic the complex functions of the human brain. These models consist of interconnected nodes or neurons that process data, learn patterns and enable tasks such as pattern recognition and decision-making.

Neural networks are capable of learning and identifying patterns directly from data without pre-defined rules. These networks are built from several key components:

- **Neurons**: The basic units that receive inputs, each neuron is governed by a threshold and an activation function.
- **Connections**: Links between neurons that carry information, regulated by weights and biases.
- **Weights and Biases**: These parameters determine the strength and influence of connections.
- **Propagation Functions**: Mechanisms that help process and transfer data across layers of neurons.
- **Learning Rule**: The method that adjusts weights and biases over time to improve accuracy.

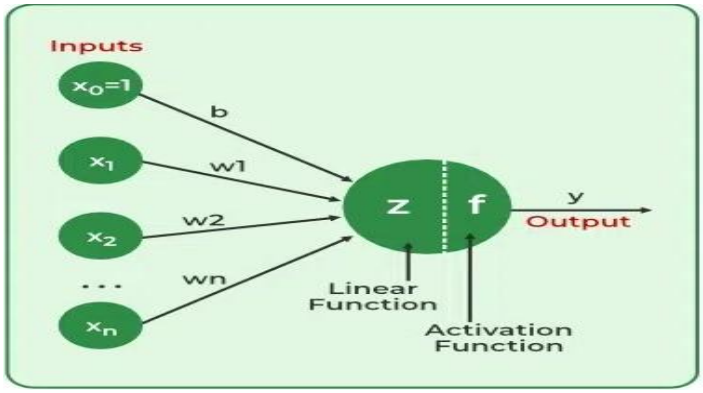
Learning in neural networks follows a structured, three-stage process:

1. **Input Computation**: Data is fed into the network.
2. **Output Generation**: Based on the current parameters, the network generates an output.
3. **Iterative Refinement**: The network refines its output by adjusting weights and biases, gradually improving its performance on diverse tasks.

In an adaptive learning environment:

- The neural network is exposed to a simulated scenario or dataset.
- Parameters such as weights and biases are updated in response to new data or conditions.
- With each adjustment, the network's response evolves allowing it to adapt effectively to different tasks or environments.

Neural Networks are a fundamental concept in artificial intelligence inspired by the structure and functioning of the human brain. They consist of layers of interconnected nodes called neurons, where each neuron processes input data using weights and bias and passes the result to the next layer. Neural networks are capable of learning complex patterns from data and are widely used in image recognition, speech processing, and natural language processing. They include an input layer, one or more hidden layers, and an output layer. The strength of neural networks lies in their ability to model non-linear relationships. As the number of layers increases, the network becomes more powerful. These deep neural networks form the basis of deep learning. Neural networks improve performance as more data becomes available. They are trained using large datasets and optimization techniques.



Working of Neural Networks

1. Forward Propagation

When data is input into the network, it passes through the network in the forward direction, from the input layer through the hidden layers to the output layer. This process is known as forward propagation. Here's what happens during this phase:

1. Linear Transformation: Each neuron in a layer receives inputs which are multiplied by the weights associated with the connections. These products are summed together and a bias is added to the sum. This can be represented mathematically as:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

where

- w represents the weights
- x represents the inputs
- b is the bias

2. Activation: The result of the linear transformation (denoted as z) is then passed through an activation function. The activation function is crucial because it introduces non-linearity into the system, enabling the network to learn more complex patterns. Popular activation functions include ReLU, sigmoid and tanh.

2. Backpropagation

After forward propagation, the network evaluates its performance using a loss function which measures the difference between the actual output and the predicted output. The goal of training is to minimize this loss. This is where backpropagation comes into play:

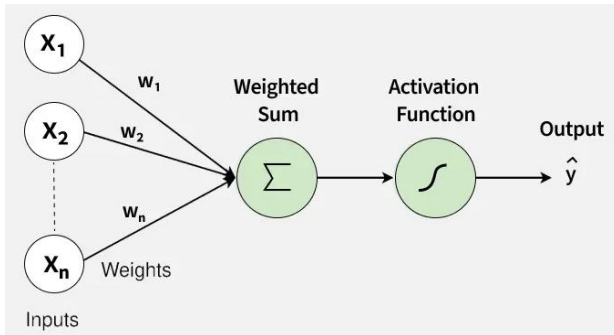
- Loss Calculation: The network calculates the loss which provides a measure of error in the predictions. The loss function could vary; common choices are mean squared error for regression tasks or cross-entropy loss for classification.
- Gradient Calculation: The network computes the gradients of the loss function with respect to each weight and bias in the network. This involves applying the chain rule of calculus to find out how much each part of the output error can be attributed to each weight and bias.
- Weight Update: Once the gradients are calculated, the weights and biases are updated using an optimization algorithm like stochastic gradient descent (SGD). The weights are adjusted in the opposite direction of the gradient to minimize the loss. The size of the step taken in each update is determined by the learning rate.

3. Iteration

This process of forward propagation, loss calculation, backpropagation and weight update is repeated for many iterations over the dataset. Over time, this iterative process reduces the loss and the network's predictions become more accurate.

Through these steps, neural networks can adapt their parameters to better approximate the relationships in the data, thereby improving their performance on tasks such as classification, regression or any other predictive modeling.

Perceptron is the simplest form of a neural network and acts as a basic building block for more complex models. It is a single-layer neural network used mainly for binary classification problems. A perceptron takes multiple input values, multiplies them by corresponding weights, adds a bias, and applies an activation function to produce an output. If the output crosses a threshold, the perceptron activates. Perceptrons are limited to solving linearly separable problems. They cannot handle complex patterns or non-linear data. Despite this limitation, perceptrons are important for understanding how neural networks work. They introduced the concept of weighted inputs and learning through error correction. Modern neural networks are built by combining many perceptrons. Thus, the perceptron forms the foundation of neural network learning.



1. Inputs $(x_1, x_2, \ldots, x_n)$

These are the features or measurable attributes of a data point that the perceptron uses to make a decision. Each input provides a signal that contributes to the final output.

- Example: For an OR gate, the inputs are binary: $(x_1, x_2) \in \{0,1\}^2$
- Inputs themselves have no inherent influence unless multiplied by weights.

2. Weights $(w_1, w_2, \ldots, w_n)$

Weights determine how strongly each input contributes to the prediction. A larger weight means the corresponding input has a higher impact.

- Weights are learned during training, adjusting based on errors.
- They act like importance scores for each feature.

3. Bias (b)

The bias is a constant value added to the weighted sum to shift the decision boundary.

- It allows the perceptron to classify correctly even when all input features are zero.
- Bias ensures the model is not forced to pass the decision boundary through the origin.

Difference Between Weights and Bias

- Weights control how much each input influences the output.
- Bias controls when the perceptron activates, independent of any input.
- Mathematically, Weights tilt the line and Bias shifts the line up/down or left/right.

4. Net Input (Weighted Sum)

This is the combined effect of all inputs and their weights:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- Represents the activation strength before passing through the activation function.

- If z is high or low enough, it determines the final class.

5. Activation Function (Step Function)

The activation function converts the numerical input into a binary output:

$$\hat{y} = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

- It introduces non-linearity in the decision-making, although the decision boundary remains linear.
- Output is always 0 or 1 making perceptrons suitable for binary classification.

Types of Neural Networks

There are seven types of neural networks that can be used.

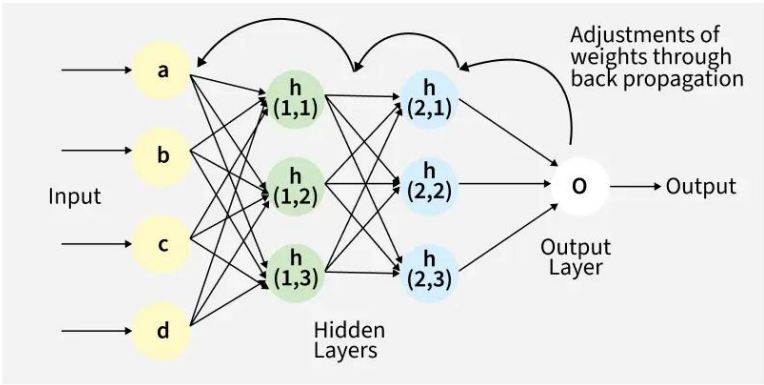
- Feedforward Networks: It is a simple artificial neural network architecture in which data moves from input to output in a single direction.
- Singlelayer Perceptron: It has one layer and it applies weights, sums inputs and uses activation to produce output.
- Multilayer Perceptron (MLP): It is a type of feedforward neural network with three or more layers, including an input layer, one or more hidden layers and an output layer. It uses nonlinear activation functions.
- Convolutional Neural Network (CNN): It is designed for image processing. It uses convolutional layers to automatically learn features from input images, enabling effective image recognition and classification.
- Recurrent Neural Network (RNN): Handles sequential data using feedback loops to retain context over time.
- Long Short-Term Memory (LSTM): A type of RNN with memory cells and gates to handle long-term dependencies and avoid vanishing gradients.

Training Process

Backpropagation

It is a key algorithm used to train neural networks by minimizing the difference between predicted and actual outputs. It works by propagating errors backward through the network, using the chain rule of calculus to compute gradients and then iteratively updating the weights and biases. Combined with optimization techniques like gradient descent, backpropagation enables the model to reduce loss across epochs and effectively learn complex patterns from data.

Backpropagation is the core training process used in neural networks to minimize prediction errors. It works by calculating the error at the output layer and propagating it backward through the network. During this process, gradients are computed to understand how much each weight contributed to the error. These gradients are then used to update the weights using optimization methods like gradient descent. Backpropagation helps the network learn by continuously adjusting weights. It allows deep networks with many layers to be trained efficiently. Without backpropagation, training neural networks would be extremely difficult. The process is repeated over multiple iterations called epochs. With each epoch, the model improves its accuracy.



Back Propagation plays a critical role in how neural networks improve over time. Here's why:

1. Efficient Weight Update: It computes the gradient of the loss function with respect to each weight using the chain rule making it possible to update weights efficiently.
2. Scalability: The Back Propagation algorithm scales well to networks with multiple layers and complex architectures making deep learning feasible.
3. Automated Learning: With Back Propagation the learning process becomes automated and the model can adjust itself to optimize its performance.

1. Forward Pass Work

In forward pass the input data is fed into the input layer. These inputs combined with their respective weights are passed to hidden layers. For example in a network with two hidden layers (h1 and h2) the output from h1 serves as the input to h2. Before applying an activation function, a bias is added to the weighted inputs.

Each hidden layer computes the weighted sum (`a`) of the inputs then applies an activation function like ReLU (Rectified Linear Unit) to obtain the output (`o`). The output is passed to the next layer where an activation function such as softmax converts the weighted outputs into probabilities for classification.

2. Backward Pass

In the backward pass the error (the difference between the predicted and actual output) is propagated back through the network to adjust the weights and biases. One common method for error calculation is the Mean Squared Error (MSE) given by:

$$\text{MSE} = (\text{Predicted Output} - \text{Actual Output})^2$$

Once the error is calculated the network adjusts weights using gradients which are computed with the chain rule. These gradients indicate how much each weight and bias should be adjusted to minimize the error in the next iteration. The backward pass continues layer by layer ensuring that the network learns and improves its performance. The activation function through its derivative plays a crucial role in computing these gradients during Back Propagation.

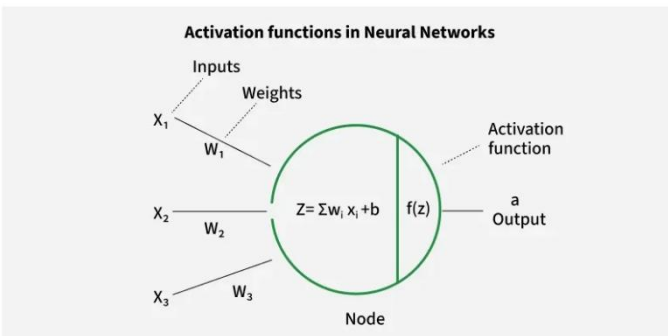
Activation Functions

An activation function in a neural network is a mathematical function applied to the output of a neuron. It introduces non-linearity, enabling the model to learn and represent complex data

patterns. Without it, even a deep neural network would behave like a simple linear regression model.

Activation functions decide whether a neuron should be activated based on the weighted sum of inputs and a bias term. They also make backpropagation possible by providing gradients for weight updates.

Activation Functions introduce non-linearity into neural networks, enabling them to learn complex patterns. Without activation functions, neural networks would behave like simple linear models. The **ReLU (Rectified Linear Unit)** function outputs zero for negative inputs and the input itself for positive values, making it computationally efficient and widely used in hidden layers. The **Sigmoid** function maps input values between 0 and 1, making it suitable for binary classification problems. However, sigmoid can suffer from vanishing gradient issues. The **Softmax** function converts outputs into probability distributions that sum to one. It is commonly used in multi-class classification tasks. Each activation function has a specific use case. Choosing the right activation function improves learning efficiency. Activation functions play a critical role in neural network performance.



Why Non-Linearity is Important

- Real-world data is rarely linearly separable.
- Non-linear functions allow neural networks to form curved decision boundaries, making them capable of handling complex patterns (e.g., classifying apples vs. bananas under varying colors and shapes).
- They ensure networks can model advanced problems like image recognition, NLP and speech processing.

Mathematical Example

Consider a neural network with:

- Inputs: i_1, i_2
- Hidden layer: neurons h_1 and h_2
- Output layer: one neuron (output)
- Weights: w_1, w_2, w_3, w_4, w_5, w_6
- Biases: b_1 for hidden layer, b_2 for output layer

The hidden layer outputs are:

- $h_1 = i_1.w_1 + i_2.w_3 + b_1$

- $h_2 = i_1.w_2 + i_2.w_4 + b_2$

The output before activation is:

- output $= h_1.w_5 + h_2.w_6 +$ bias

Without activation, these are linear equations.
To introduce non-linearity, we apply a sigmoid activation:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

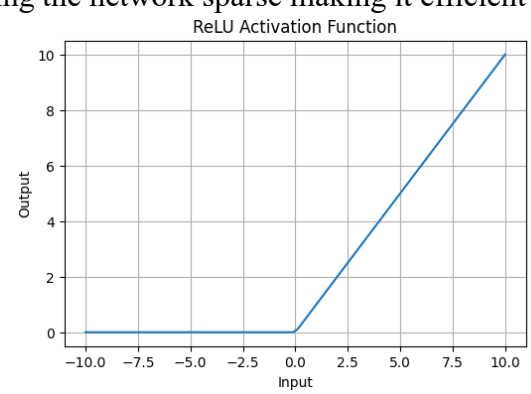
- final output $= \sigma(h_1.w_5 + h_2.w_6 +$ bias$)$

This gives the final output of the network after applying the sigmoid activation function in output layers, introducing the desired non-linearity.

ReLU (Rectified Linear Unit) Function

ReLU activation is defined by $A(x) = \max(0, x)$, this means that if the input x is positive, ReLU returns x, if the input is negative, it returns 0.

- Value Range: $[0, \infty)$, meaning the function only outputs non-negative values.
- Nature: It is a non-linear activation function, allowing neural networks to learn complex patterns and making backpropagation more efficient.
- Advantage over other Activation: ReLU is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.



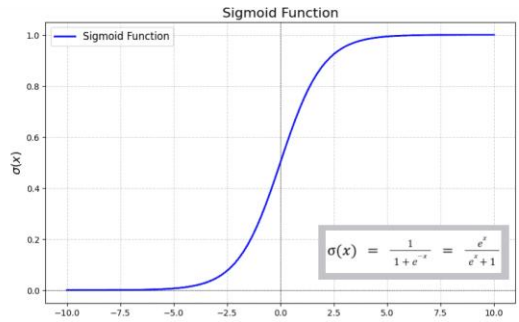
Output Layer Activation Functions

Sigmoid Activation Function

Sigmoid function produces an S-shaped curve and maps input values into a probability-like range between 0 and 1 and is used to find the final output of the neural network for binary classification problems. It is defined as:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

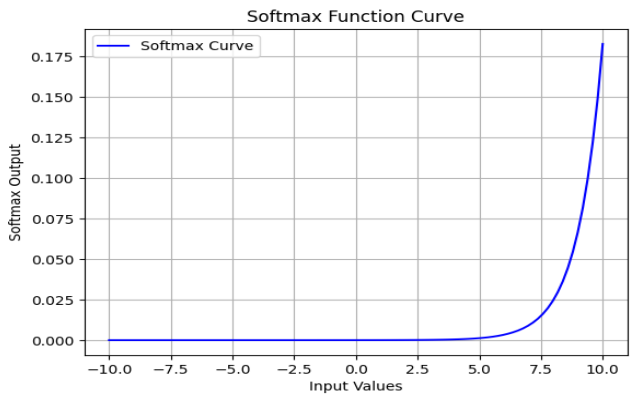
- Value Range: (0,1)(0,1)(0,1)
- Smooth and Differentiable: Useful for gradient-based optimization.
- Best for: Binary classification output layers.
- Interpretable Output: Can be treated as probability.
- Limitation: Suffers from vanishing gradient for very large or very small inputs.



Softmax Function

Softmax function is designed to handle multi-class classification problems. It transforms raw output scores from a neural network into probabilities. It works by squashing the output values of each class into the range of 0 to 1 while ensuring that the sum of all probabilities equals 1.

- Softmax is a non-linear activation function.
- The Softmax function ensures that each class is assigned a probability, helping to identify which class the input belongs to.



PyTorch

PyTorch is a deep learning library built on Python. It provides GPU acceleration, dynamic computation graphs and an intuitive interface for deep learning researchers and developers. PyTorch follows a "define-by-run" approach meaning that its computational graphs are constructed on the fly allowing for better debugging and model customization.

Key Features of PyTorch

- It uses dynamic graphs allowing flexibility in model execution and debugging.
- It provides an automatic differentiation engine that simplifies gradient computation for deep learning.
- It supports CUDA allowing computations to be performed efficiently on GPUs.

PyTorch is a popular open-source deep learning framework widely used for building and training neural networks. It is known for its dynamic computation graph, which allows flexibility and ease of debugging. PyTorch provides strong support for tensor operations and automatic differentiation. It is highly preferred in research and academia due to its Pythonic syntax. PyTorch supports GPU acceleration, making it suitable for large-scale deep learning tasks. It includes modules for neural network construction, optimization, and loss functions. The framework integrates well with NumPy and other Python libraries.